

Flight Booking Website — Explanation (Part 4/5)

Part 4: Frontend structure, shared JS, and page behaviors.

1) Frontend architecture

The frontend is a collection of HTML pages under `frontend/`. They share styling via `frontend/css/style.css` and JavaScript via `frontend/js/*.js`.

- Navigation is by full page loads (no SPA router).
- State is stored in browser storage (usually `localStorage`).
- Backend is called via `fetch()` returning JSON.

2) Styling: `frontend/css/style.css`

This is the central stylesheet. It defines:

- Layout (headers/nav, containers, cards)
- Form styling (inputs, buttons, validation messages)
- Tables and dashboard styles
- Responsive rules for smaller screens

3) Shared API wrapper: `frontend/js/api.js`

This file centralizes how the browser talks to the backend. It commonly provides:

- **Base URL:** a string for the backend API root.
- **request helper:** sets headers, serializes JSON, parses JSON response.
- **Modules:** `AuthAPI`, `FlightsAPI`, `BookingsAPI`, `AirportsAPI`, etc.

Important mismatch: parts of this wrapper target “World B” endpoints (airports/airlines/seat classes). Some of the dashboard pages target “World A” endpoints (company/passenger schema).

4) Main UI logic: frontend/js/main.js

This file typically attaches event listeners to forms/buttons and implements page behaviors. Examples of common logic:

- Handle search form submit (redirect to results page with query params).
- Load dynamic lists like cities/airports (calling backend endpoints).
- Read current user from `localStorage` to personalize UI.

5) Core pages

Page	Purpose
<code>index.html</code>	Landing page + entry point to search / login.
<code>login.html</code>	Login form; on success stores user info and redirects by user type.
<code>register.html</code>	Register form; chooses passenger vs company and posts to register endpoint.

6) Passenger pages (World A)

Page	What it does
<code>passenger-dashboard.html</code>	Shows passenger bookings list and actions.
<code>passenger-profile.html</code>	Shows/edit passenger profile; upload photo/passport; change password.
<code>search-results.html</code>	Shows flights found from a search; usually calls search endpoint.
<code>flight-info.html</code>	Displays one flight details and booking actions.

7) Company pages (World A)

Page	What it does
<code>company-dashboard.html</code>	Lists company flights and links to details.

Page	What it does
<code>add-flight.html</code>	Create a flight + define itinerary (cities sequence).
<code>flight-details.html</code>	Flight details for company; lists pending/registered passengers.
<code>company-profile.html</code>	Update company profile data (bio/logo/etc.).

8) Messaging pages

Passenger and company both have pages to view conversations and send messages. Those pages call `backend/api/messages.php`.

9) Files covered in Part 4

- `frontend/css/style.css`
- `frontend/js/api.js`
- `frontend/js/main.js`
- All HTML pages under `frontend/` (login/register, passenger/company dashboards, flight/search pages, profiles, messaging).

End of Part 4/5.